SCSE2133
Introduction to Software Project Management (SPM)

Rohayanti Hassan
Faculty of Computing
Universiti Teknologi Malaysia

This slide contains material that is copied from the book Agile Project Manager_v2_handbook by Agile Business Consortium. All rights reserved. No part of this material may be reproduced or transmitted in any form or by any means without the prior written permission of the copyright owner. This material is used here for educational purposes under the doctrine of fair use, with the intention to support learning and teaching.

For more information, please refer to the original source:
Book Title: Agile Project Manager_v2_handbook
Author: Agile Business Consortium
Publisher: Buckland Media Group Limited
Year of Publication: 2017
Please consult the book for further details and complete content.

innovative ● entrepreneurial ● global

Course Synopsis

This course provides an intermediate-level introduction to **Software Project Management (SPM)** with a focus on **agile methodologies**. It covers fundamental concepts, challenges, and management controls in SPM, followed by agile principles such as project identification, analysis, planning, documentation, and user stories. Students explore frameworks including the **Dynamic Systems Development Method (DSDM), Scrum, Kanban, and Scrumban, along with project management tools and practices like MoSCoW prioritization, timeboxing, and daily stand-ups**. The DSDM life cycle from pre-project to post-project phases is examined in detail, alongside agile planning, control, and testing concepts. Taught through lectures, hands-on labs, and industry-based case studies, the course emphasizes collaboration, problem-solving, and project-driven learning. By the end, students will be able to **analyze and apply** SPM principles, design agile project plans, evaluate and adapt agile frameworks, and implement effective prioritization and time management strategies to deliver successful software projects.

	Continous Assessment	%
1	Quiz 1	5
2	Project Assignment 1- User Stories	15
3	Project Assignment 2 - Moscow Classification	10
4	Project Assignment 3 - Time- Boxing	15
5	Project Assignment 4 - Monetization Planning	10
6	Mid Semester exam	15

	Summative Assessment	%
1	Final Exam	30

Outline

- The Fundamental Concepts of SPM
- The Challenges in SPM
- The Concepts of Management Control in SPM



The Fundamental Concepts of SPM

What is software project management ?

Software Project Management is both an art and a discipline. It involves:

- Planning and supervision of software projects, applying structured methods and techniques.
- Managing resources, including allocation and timing, to ensure that the final software fulfills the defined requirements.
- Providing clarity on project parameters—such as duration, timeline, and cost—for both clients and developers.

Why Is Software Project Management Important?

Strategic alignment



Ensures that software efforts are in harmony with organizational goals.

Clear priorities and objectives



Sets direction and measurable endpoints.

Strong leadership



Guides teams toward focused results

Realistic project planning



Promotes feasible scheduling and resource allocation.

Why Is Software Project Management Important?

Quality control



Upholds standards throughout development.

Learning from outcomes



Uses successes and failures to continuously improve processes.

Risk management



Anticipates and mitigates uncertainties.

Advantages of Software Project Management

- Facilitates the planning of software development, creating a clear roadmap from conception to delivery.
- Simplifies the implementation phase, ensuring tasks are executed efficiently and in the right order.
- Enables effective monitoring and control, helping teams stay aligned and adapt as needed.
- Ultimately helps save time and costs, reducing waste and enhancing resource utilization.

Disadvantages of Software Project Management

- Can introduce complexity, making tasks harder to understand—especially for those unfamiliar with project management tools or formal processes.
- Involves potential costs, such as investing in high-quality project management software or training team members.
- Some team members may face learning challenges, struggling to adapt to new tools or methodologies.

Software Project Planning

- **Define the Scope of Work**
  Clearly outline what tasks and deliverables the software project includes. This helps prevent confusion and scope creep later.

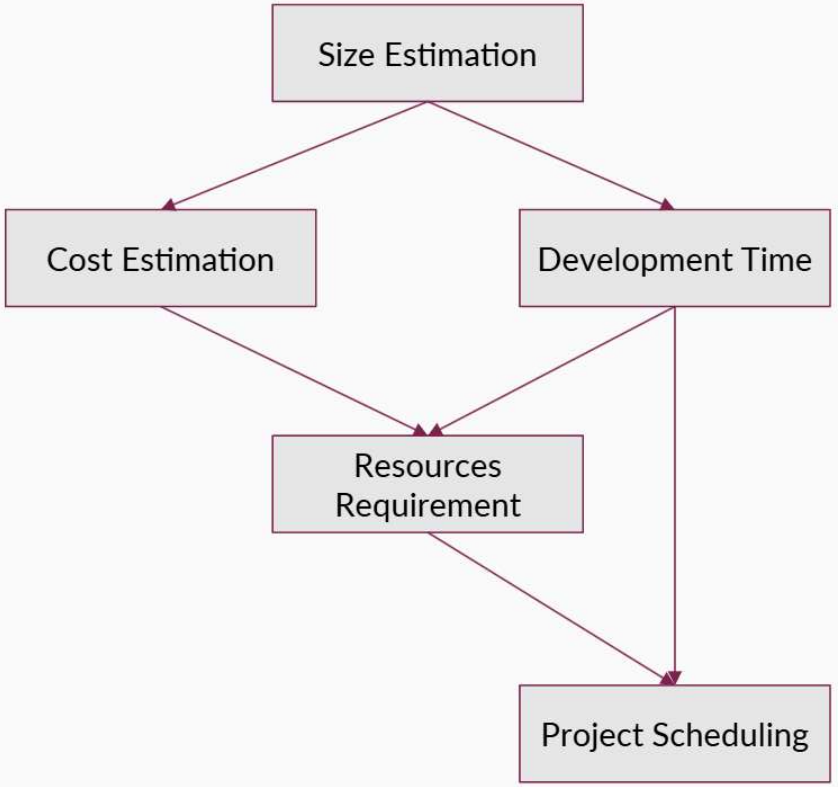
- **Conduct Risk Analysis**
  Identify potential risks (technical, resource-related, timeline issues) that could impact the project, and plan mitigation strategies.

- **Determine Required Resources**
  List all mandatory resources, including manpower, tools, equipment, and budget needed for successful completion.

Software Project Planning

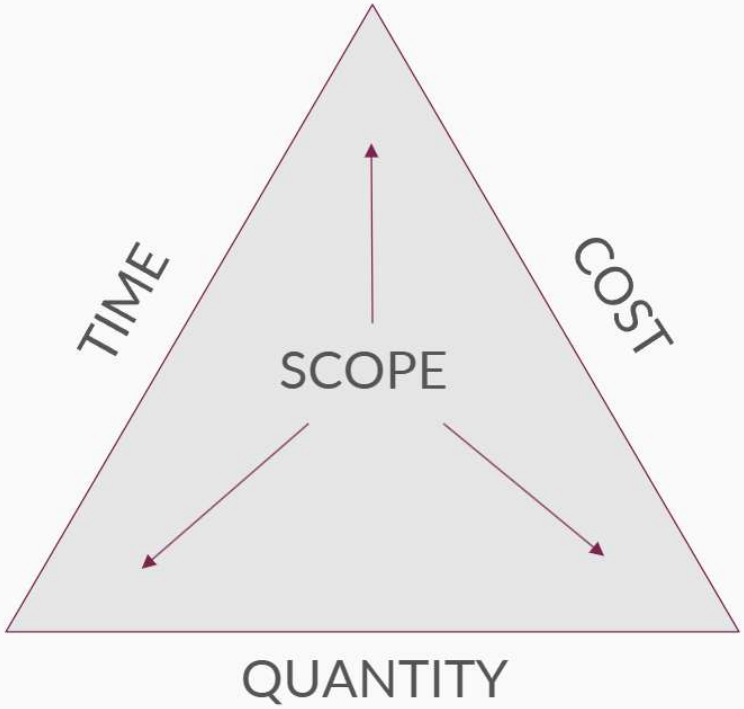
- **Set Project Goals and Deliverables**
  Define what the project aims to accomplish, along with measurable success criteria and final deliverables.

- **Maintain Proper Documentation**
  Keep a record of the entire planning process, including decisions, assumptions, estimations, and timelines, to ensure traceability and accountability.



Needs for Software Project Management

Software Project Management (SPM) is essential to balance the core constraints of any software project:



Needs for Software Project Management

⏱️ Time

- Refers to the project schedule and deadlines.
- Delays can increase costs and reduce stakeholder satisfaction.

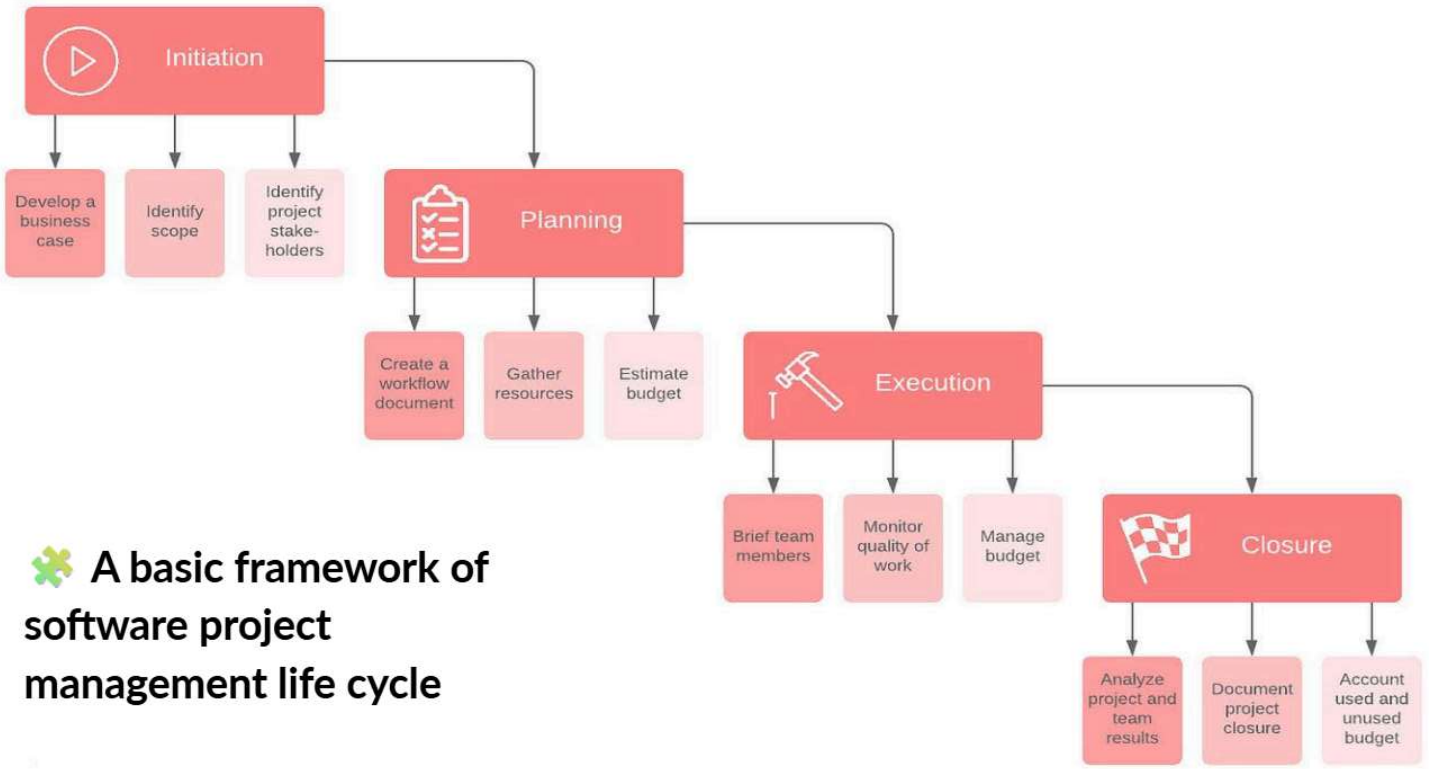
💲 Cost

- Involves budgeting, resource allocation, and financial control.
- Overspending can derail the project or reduce profit margins.

✅ Quality

- The degree to which the software meets requirements and user expectations.
- Rushing or underfunding a project may reduce quality.

Software Project Management Life Cycle



🧩 **A basic framework of software project management life cycle**

Software Project Management Life Cycle

🧩 What Is the Software Project Management Life Cycle?

The **Software Project Management Life Cycle** is a structured framework that guides a software project from **beginning to end**. It ensures that every phase of the project — from concept to delivery — is **planned, monitored, and controlled** to meet objectives like time, cost, scope, and quality.

🟦 General Definition:

Software Project Management Life Cycle refers to the sequence of phases a software project goes through, from initiation to closure, to ensure successful planning, execution, and delivery.

Software Project Management Life Cycle

🛠️ Why It's Important:

- Provides a **clear roadmap** for managing complex software projects.
- Helps teams stay aligned with **goals, deadlines, and budgets**.
- Ensures **risk identification**, proper **resource allocation**, and **quality control**.
- Promotes **accountability** by defining roles and deliverables at each stage.

Software Project Management Life Cycle

Initiation Phase

Objective: To define the project at a high level and secure approval to begin.

- **Key Activities:**

- **Assess Business Needs**
  Understand what problem the software aims to solve.
- **Feasibility Study**
  Analyze whether the project is technically and financially viable.
- **Identify Stakeholders**
  Who will benefit from or be involved in the project?
- **Develop Project Charter**
  A formal document outlining objectives, scope, and high-level plan.

Software Project Management Life Cycle

Planning Phase

Objective: To create a roadmap for how the project will be executed, monitored, and closed.

- **Key Activities:**

- **Define Scope in Detail**
  What is included and excluded from the project.
- **Set Project Objectives**
  Clear and measurable goals.
- **Estimate Resources & Budget**
  How much time, money, and manpower is needed?

Software Project Management Life Cycle

- **Schedule Activities**
  Use Gantt Charts, Milestones, or Sprints to plan the timeline.
- **Plan for Risks**
  Identify potential obstacles and backup strategies.
- **Communication Plan**
  Decide how progress will be reported and to whom.

Software Project Management Life Cycle

Execution Phase

Objective: To actually carry out the work defined in the plan.

- **Key Activities:**

- **Develop Software**
  Coding, designing, and building features.
- **Conduct Testing**
  Unit tests, integration tests, and QA cycles.
- **Manage Quality**
  Ensure standards and client expectations are met.
- **Track Progress**
  Regular status updates, meetings, and monitoring.

Software Project Management Life Cycle

Closure Phase

Objective: To formally close the project and evaluate its success.

- **Key Activities:**

- **Final Testing & Approval**
  Client or stakeholder acceptance of deliverables.
- **Release Product**
  Deploy or hand over the software to the client.
- **Team Release & Handoff**
  Return resources, and transfer to maintenance team if needed.
- **Postmortem Review**
  Lessons learned, success metrics, and process evaluation.

Project Success Criteria

What Defines a Successful Software Project?

To evaluate whether a software project is successful, consider the following key criteria:

- **Meeting Requirements**
  The software delivers all specified features and functionality.

- **Stakeholder Satisfaction**
  Clients, users, and other stakeholders are happy with the result.

- **On-Time Delivery**
  The project is completed within the scheduled time frame.

Project Success Criteria

- **Within Budget**
  The project stays within its allocated financial resources.

- **Product Quality**
  The software performs reliably, is free of critical bugs, and meets quality standards.

- **Maintainability & Scalability**
  The system is easy to maintain, update, and scale for future needs

Common Tools in Software Project Management

Tools That Support Project Planning, Execution & Control

These tools help managers and teams streamline tasks, manage timelines, and track progress:

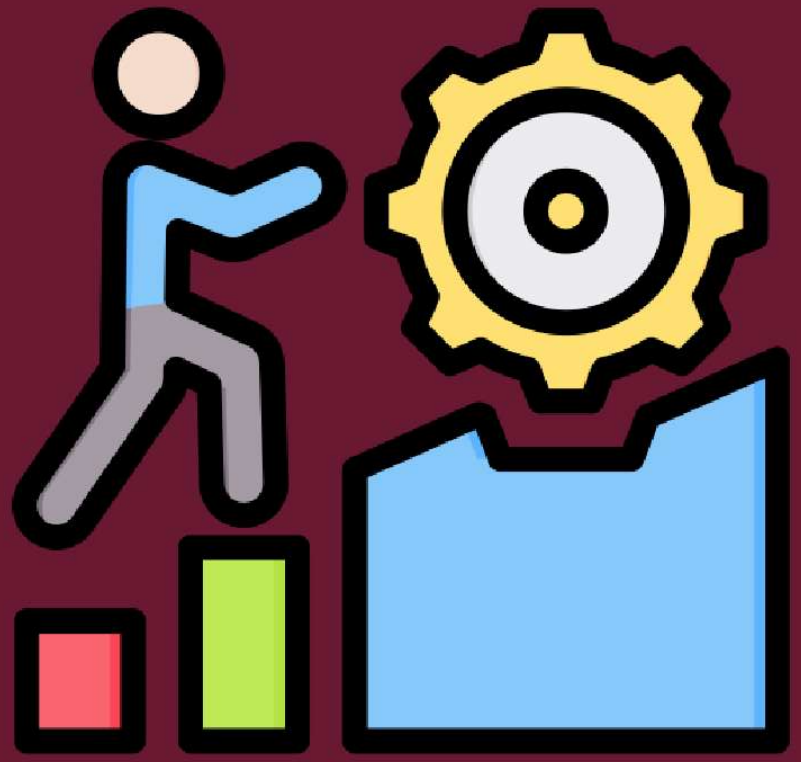
Tools	Purpose
Gantt Charts	Visual project timelines and task dependencies
Jira	Agile project tracking (issues, sprints, boards)
Microsoft Project	Detailed planning and resource allocation
Trello	Visual Kanban boards for task tracking
Slack / Teams	Communication & collaboration

🔍 Choose tools based on project size, methodology (Agile vs Waterfall), and team needs.

Summary of Fundamental Concepts

Summary of Fundamental Concepts

- Software Project Management (SPM) is the **structured discipline** of planning, leading, and controlling software projects.
- It aims to align **technical execution** with **business goals** using defined methods and tools.
- Core aspects include:
  - Project planning (scope, resources, risks)
  - Process models and life cycles
  - Time–Cost–Quality–Scope balance
  - Use of project management tools (Jira, Gantt, EVM)
  - Success criteria like stakeholder satisfaction and product quality



The Challenges in SPM

Challenge in Software Project Management

1. **Manager's Point of View – Challenges**

- Poor estimation and unrealistic planning
- Lack of quality standards or performance metrics
- Limited guidance for decision-making
- No clear visibility into project progress
- Undefined roles and responsibilities
- Incorrect or unclear success criteria

Challenge in Software Project Management

2. Team Member's Point of View – Challenges

- Vague or incomplete work specifications
- Managers not understanding IT processes
- Lack of domain knowledge (application area)
- Absence of development and quality standards
- No clear quality expectations
- Limited involvement in early planning

Challenge in Software Project Management

3. Technical & Documentation Issues

- Lack of updated or complete documentation
- Technical deliverables delayed by unfinished dependencies
- Poor communication between teams and users
- Duplication of work due to communication breakdown
- Outdated technical references or guidelines
- No centralized document management system

Challenge in Software Project Management

4. Commitment & Ownership Problems

- Low commitment from key personnel
- Over-reliance on one team member
- Team members moved before project completion
- Lack of ownership mindset
- Inconsistent accountability tracking
- Minimal recognition for contributions

Challenge in Software Project Management

5. Expertise & Resource Limitations

- Narrow technical skill sets in the team
- Inadequate training for modern tools or technologies
- Lack of backup staff with required competencies
- Insufficient onboarding for new team members
- Mismatch between roles and expertise
- Resource availability not properly planned

Challenge in Software Project Management

6. Changing External Conditions

- Evolving statutory/legal requirements
- Rapidly changing software environments
- Frequent updates in third-party platforms
- Unclear adaptation processes for change
- Vendor dependency risks
- Compliance challenges across regions

Challenge in Software Project Management

7. Time Pressure & Deadlines

- Constant pressure to meet tight deadlines
- Testing phase often shortened or skipped
- Frequent last-minute changes
- Developer burnout and overtime
- Reduced time for proper documentation
- Compromised quality due to speed

Challenge in Software Project Management

8. Quality & Control Issues

- Inadequate quality control processes
- No standard for code reviews or testing
- Bugs detected late in development
- Inconsistent QA practices
- Lack of automated testing tools
- Unclear defect tracking procedures

Challenge in Software Project Management

9. Modern Working Challenges

- Remote management and lack of real-time collaboration
- Time zone differences causing coordination issues
- Low team morale in virtual settings
- Over-reliance on asynchronous tools
- Difficulty building team rapport online
- Lack of real-time collaboration channels

Summary of Key Challenges in SPM

- Poor estimation and vague specifications
- Limited documentation and communication breakdowns
- Scope creep and unclear success criteria
- Skill/resource mismatches and over-dependency on individuals
- Time pressure, quality issues, and adapting to remote work environments



The Concepts of Management Control in SPM

What Is Management Control in SPM?

Management control in software project management refers to the **systematic process of guiding a project towards its objectives** by continuously monitoring progress, identifying deviations, and implementing corrective actions.

It ensures:

- The project stays **on schedule**, **within budget**, and **aligned with scope**.
- Proper **feedback mechanisms** are in place to respond to issues quickly.
- The team and stakeholders stay **accountable and informed** throughout the project.

Why Is Management Control Important?

Without proper control, a project can easily go off track. Here's why it matters:

- Prevents **time and budget overruns**
- Helps **manage change** effectively
- Improves **decision-making** through data and tracking
- Promotes **accountability** among teams
- Ensures **quality standards** are maintained
- Builds trust with stakeholders by providing transparency

Effective control is not about micromanaging — it's about steering the project toward success with foresight and agility.

Key Elements of Control in SPM

Management control can be broken down into **six core focus areas**:

1. **Time Control** – Managing schedules and deadlines
2. **Cost Control** – Monitoring budgets and expenditures
3. **Scope Control** – Ensuring agreed deliverables are met
4. **Risk Control** – Identifying and mitigating risks early
5. **Quality Control** – Meeting technical and user expectations
6. **Communication Control** – Keeping all parties informed

Each of these elements must be regularly reviewed and adjusted based on real-time project performance.

1. Time Control

Time is one of the most critical constraints in any project. Time control includes:

- **Creating schedules** with start and end dates
- Breaking work into **tasks and milestones**
- Using tools like **Gantt charts**, **Kanban boards**, or **Sprint backlogs**
- Holding **daily stand-ups** or weekly syncs to check progress
- Reallocating resources if delays are forecasted
- Using **timeboxing** techniques in Agile to enforce strict timelines

A delay in one task often impacts many others — hence tight time control is key.

2. Cost Control

Keeping spending in check is essential to avoid budget overruns. This involves:

- Estimating costs during planning and **tracking actual expenses** during execution
- Performing **cost variance analysis** to detect overspending early
- Using **Earned Value Management (EVM)** for integrated cost/schedule control
- Approving only **authorized expenditures**
- Updating stakeholders if a budget change is unavoidable

Even small unexpected costs can escalate quickly without proper control.

3.Scope Control

Scope refers to the **total set of deliverables** expected. Without scope control:

- You risk **scope creep** (adding features without adjusting time or cost)
- You may deliver something **different from what was agreed**

Scope control techniques include:

- Using a **Work Breakdown Structure (WBS)**
- Tracking changes through a **Change Control Board (CCB)**
- Regularly **validating deliverables** against original requirements
- Maintaining a **Requirements Traceability Matrix**

4.Risk Management Control

Risks are inevitable. Management control involves:

- Identifying **potential risks** early through brainstorming or checklists
- Evaluating them based on **likelihood and impact**
- Creating **mitigation plans** for high-risk scenarios
- Keeping a **risk register** with status updates
- Reassessing risks regularly during the project lifecycle

A proactive approach to risk reduces firefighting and last-minute panic.

5.Issue Tracking & Escalation

While risks are hypothetical, **issues are problems that already occurred**.

Good issue control means:

- Logging issues in a centralized **issue tracker**
- Assigning **clear ownership** for resolution
- Setting **deadlines** and **escalation paths** if unresolved
- Reviewing issues during regular team or stakeholder meetings
- Analyzing root causes to avoid recurrence

Unmanaged issues lead to delays, conflict, and dissatisfaction.

6.Quality Control

Delivering software that works isn't enough — it must work well. Quality control ensures:

- Compliance with **technical standards** and **coding guidelines**
- Execution of **testing procedures** (unit, integration, system, UAT)
- Reviews such as **peer code reviews**, **walkthroughs**, and **inspections**
- **Tracking defects** and resolution cycles
- Tools like **SonarQube, Selenium**, or manual testing methods

Quality control is about *doing it right the first time*.

7.Communication Control

Effective communication is the glue that holds everything together.

Control mechanisms include:

- A **communication plan** defining what gets reported, to whom, and how often
- Tools like **Slack, Teams, Email, Jira** for ongoing coordination
- **Stakeholder updates** (status reports, dashboards, presentations)
- **Meeting schedules** and minutes to document decisions
- Avoiding over-communication (noise) and under-communication (blind spots)

Transparency and clarity reduce confusion, rework, and frustration.

Monitoring Tools in SPM

You can't control what you don't measure. Popular tools include:

- **Gantt Charts** – visualize timelines and task dependencies
- **Burndown Charts** – monitor progress in Agile
- **Project Dashboards** – track KPIs and performance
- **Jira / Trello / Asana** – manage tasks and workflows
- **Time Tracking Tools** – for cost/time analysis
- **Issue Logs & Risk Registers** – for governance

Earned Value Management (EVM)

Earned Value Management (EVM)

EVM helps you **quantitatively assess project performance**. Key terms:

- **Planned Value (PV)** – work planned up to a point
- **Earned Value (EV)** – work actually completed
- **Actual Cost (AC)** – money spent so far

Using EVM, you can calculate:

- **Schedule Variance (SV)** = EV - PV
  Cost Variance (CV) = EV - AC

Positive variance = on track. Negative = corrective action needed.

Change Control Process

Change is inevitable. Managing it effectively includes:

- A **formal process** for submitting change requests (CRs)
- Analyzing impact on **time, scope, and budget**
- Decision-making by a **Change Control Board (CCB)**
- Documenting all approved or rejected changes
- Communicating outcomes clearly to affected parties

Good change control avoids chaos and uncontrolled growth.

Key Performance Indicators (KPIs)

KPIs help you **quantify performance**. Examples include:

- % of completed tasks vs planned
- Number of open vs closed bugs
- Sprint velocity (Agile)
- Burn rate (cost over time)
- Customer satisfaction or NPS
- Time-to-market for features

Choose KPIs that align with your project goals and update them regularly.

Summary & Best Practices

To wrap up:

- Use **real-time monitoring** to control deviations
- Focus on **preventive actions**, not reactive fixes
- Communicate consistently and clearly
- Balance flexibility with control
- Leverage **tools and dashboards** effectively

Strong management control doesn't slow down projects — it makes success repeatable and reliable.



THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

www.utm.my